SIGSM

Construiremos, desde cero, el módulo de gestión documental del Hospital de Clínicas.

Cada concepto que veamos se aplica inmediatamente a este caso real. El cliente pidió digitalizar indicaciones médicas (IVE, imagenología, nefrología, etc.) agrupadas por categoría, accesibles públicamente. Al final de las 3 semanas tendrán:

- Cliente estático HTML+CSS+JS
- API PHP que devuelve JSON
- MySQL con 3 tablas mínimas
- Subida de PDF, listado, edición y baja lógica
- Login con localStorage y rol admin/usuario
- Vista pública por categorías (mobile-first)

USUARIOS
`id · email · password_hash · rol · creado_en`

CATEGORIAS
`id · nombre · descripcion · creada_en`

DOCUMENTOS
`id · titulo · descripcion · categoria_id · ruta_archivo · activo · creado_en · actualizado_en`

Arquitectura 3 capas

Simplificada — lo justo para esta entrega

PRESENTACIÓN

HTML + CSS + JS

- Carpeta: /public
- Cliente estático servido por Apache (htdocs / var/www/html)
- JS hace fetch a /sigsm/api/...
- Nunca incluye lógica de negocio

NEGOCIO

PHP

- Carpeta: /api
- Un archivo PHP por endpoint
- Lee $_POST, $_FILES, $_SERVER
- Devuelve SIEMPRE JSON
- Nunca renderiza HTML

DATOS

MySQL

- PDO: conexión y consultas
- DDL en /db/esquema.sql
- Datos de prueba en /db/datos_prueba.sql
- Solo fetch_assoc
- No usamos transacciones ni JOIN en esta entrega

El cliente JS hace fetch a `/sigsm/api/endpoint.php` → PHP lee la request, consulta MySQL con PDO → devuelve JSON → JS pinta el DOM. **PHP nunca imprime HTML.**

Estructura del proyecto

Cada cosa en su carpeta — armar esto antes de escribir código

ÁRBOL DEL PROYECTO

```
ÁRBOL DEL PROYECTOsigsm/
├── public/                ← cliente estático (htdocs / var/www/html)
│   ├── categorias.html
│   ├── documentos.html
│   ├── login.html
│   ├── backoffice/
│   │   └── dashboard.html
│   ├── js/
│   │   ├── cargar.js
│   │   ├── login.js
│   │   ├── categorias.js
│   │   ├── documentos.js
│   │   ├── auth-fetch.js
│   │   ├── api.js
│   │   └── backoffice/
│   │       └── dashboard.js
│   ├── css/
│   └── assets/
│
── api/ ← un archivo PHP por endpoint
│   ├── documentos_listar.php
│   ├── documentos_crear.php
│   ├── documentos_editar.php
│   ├── documentos_eliminar.php
│   ├── categorias_listar.php
│   ├── documentos_por_categoria.php
│   └── login.php
├├── includes/ ← compartido
│   ├── cors.php
│   ├── conexion.php
│   ├── Documento.php
│   ├── Categoria.php
│   └── Usuario.php
│
├── db/
│   ├── esquema.sql ← CREATE TABLE
│   └── datos_prueba.sql ← INSERT
│
├── uploads/ ← PDFs subidos (chmod 755)
├── README.md
└── .gitignore
```

PHP en 60 segundos

Lo mínimo para leer y escribir PHP — vienen de Java/JS, esto es directo

Conceptos clave para sobrevivir:

- Variables con $, sin tipo declarado
- Punto (.) concatena strings (no es +)
- Arrays asociativos: ['k' => 'v']
- Funciones: function foo($x) { ... }
- Clases: class Documento { ... } (semana 2)
- echo imprime, pero nosotros solo devolveremos JSON
- Comentarios: // o # o /* */
- Todas las sentencias terminan en ;

EJEMPLO.PHP

```php
<?php
$titulo = "Indicaciones IVE";
$doc = ['id' => 1, 'titulo' => $titulo];

function saludar($nombre) {
  return "Hola, " . $nombre;
}

echo saludar("María");      // Hola, María
echo $doc['titulo'];          // Indicaciones IVE

// Concatenación con punto
$msg = "Documento: " . $doc['titulo'];
```


Superglobales que usaremos

Variables predefinidas de PHP que reciben datos del cliente

Variable	Contiene	Ejemplo
$_GET	Query string (?id=5)	$_GET['id']
$_POST	Body de form o FormData	$_POST['titulo']
$_FILES	Archivos subidos (multipart)	$_FILES['pdf']['tmp_name']
$_SERVER	Info del request HTTP	$_SERVER['REQUEST_METHOD']

Para leer JSON crudo del body (no FormData) usamos file_get_contents('php://input') + json_decode(..., true). En esta entrega casi siempre usaremos FormData, así que alcanza con $_POST y $_FILES.